15.C51 Modeling with Machine Learning: Financial Technology

Project 2: Explainable Credit Scoring and Portfolio Backtesting

Erwin Deng Corentin Venet Thea (Zihui) Li Simos Michalopoulos

1. Executive Summary

This report presents several machine-learning models to help make credit approval decisions using the UCI Credit Approval dataset. The goal is not just to predict approvals accurately, but to test whether a model can support the full lending decision process: ranking applicants by risk, estimating approval probabilities, explaining individual decisions, and estimating the financial impact of different approval rules. Hence, explainability will be important for our models.

The best-performing model is a calibrated XGBoost classifier. On the held-out test set, it achieves an AUC-ROC of 0.965, a Gini coefficient of 0.930, a KS statistic of 0.827, and a Brier score of 0.070. Logistic regression is used as a simpler benchmark because it is closer to a traditional credit scorecard. Random forest, SVM, MLP, and one-hot XGBoost are included as challenger models. Since lending decisions depend on probability estimates, not just rankings, the scorecard and financial backtest use the calibrated XGBoost probabilities.

The main addition to the analysis is a risk-based portfolio backtest. Instead of giving every approved applicant the same loan amount, APR, and recovery assumption, the backtest assigns different credit limits, prices, and recovery rates based on the applicant's score band. Under the base risk-based policy, the profit-maximizing threshold is 0.72. At this threshold, the model approves 35.5% of applicants, with a bad-proxy rate of 4.1% among approved applicants and scaled one-period expected profit of $1,394,928 for 10,000 applications.

The main recommendation is cautious. Calibrated XGBoost is the strongest research model in this project, but we know that the dataset is too small and too anonymized for real-world deployment. Before any production use, the model would need reject-inference correction, adverse-impact testing, and out-of-time validation on a larger dataset with real loan performance outcomes. The A7 ablation is especially important. If removing A7 leads to only a small drop in performance while reducing governance risk, the safer production choice would be the model without A7.

Model	AUC	Gini	KS	Avg Precision	Brier	Log Loss	Accuracy@0.5	F1@0.5
Logistic Regression baseline	0.960	0.920	0.818	0.942	0.079	0.263	0.870	0.857
XGBoost raw	0.961	0.923	0.821	0.918	0.074	0.261	0.884	0.864
XGBoost calibrated	0.965	0.930	0.827	0.929	0.070	0.478	0.913	0.898
XGBoost one-hot robustness	0.962	0.924	0.827	0.922	0.073	0.261	0.891	0.876

Table 1: Hold-out model validation. The calibrated XGBoost model is used for scorecard and backtest probabilities; raw XGBoost is retained for SHAP explainability.

The report follows the four elements required by the project rubric: a complete financial analysis through threshold optimization, risk-based lending terms, and stress scenarios; methodological novelty through calibrated XGBoost, SHAP reason codes, scorecard construction, and fairness

ablation; readability through a structured model-validation style report; and transparent source attribution through a detailed LLM usage and prompt log.

2. Data and Universe

2.1 Data Source and Target Definition

The analysis uses the UCI Credit Approval dataset, accessed through `ucimlrepo`. The dataset contains 690 credit applications, 15 anonymized input variables, and a binary historical application outcome indicating whether the application was approved or denied. The positive class in this project is $P(\text{Approve})$, not $P(\text{Repay})$ or $P(\text{No Default})$. This distinction is central to interpreting the backtest: the dataset does not contain realized loan-performance outcomes.

The approval rate is moderately balanced, so AUC, KS, average precision, calibration, and score-band monotonicity are more informative than accuracy alone.

Item	Value
Observations	690
Features	15
Approval rate	44.5%
Total missing values	67
Train observations	552
Test observations	138

Table 2: Dataset summary generated directly from the project notebook.

2.2 Feature Anonymization and Working Labels

The UCI Credit Approval dataset deliberately anonymizes all variable names and categorical values to protect confidentiality. The original variables are labeled A1 through A15, and the category values are represented by arbitrary symbols rather than meaningful names.

For readability, we use descriptive working labels for some variables later in the report. These labels are based on commonly circulated reverse-engineered interpretations of the dataset and were cross-checked with LLM assistance. However, they should not be treated as verified ground truth. All model training, validation, scorecard construction, SHAP analysis, and financial backtesting are performed using the original anonymized variables. The descriptive labels are used only to make the discussion more interpretable.

This distinction is important because one variable, often interpreted as a demographic or ethnicity-related field, could raise fairness and regulatory concerns if that interpretation were correct. Since the dataset does not verify this mapping, we treat the feature as a potential protected-class proxy and evaluate its effect separately in the fairness-ablation analysis.

2.3 Preprocessing and Leakage Control

Imputation, scaling, and categorical encoding are placed inside `sklearn` `Pipeline` and `Column Transformer` objects. This avoids fitting preprocessing steps on the full dataset before cross-validation. Numerical variables are median-imputed and standardized. Categorical variables are ordinal-encoded for tree-based models and one-hot encoded for logistic regression, SVM, MLP, and robustness checks.

3. Model Development and Validation

3.1 Champion–Challenger Design

We evaluate logistic regression, random forest, XGBoost, SVM, and MLP. Logistic regression provides the practical scorecard benchmark. XGBoost is the main non-linear challenger because it is well suited to tabular data and compatible with SHAP explanations. MLP is included as a complexity check rather than as the expected winner, since 690 observations is a small sample for neural-network modeling.

Model	CV Accuracy	CV AUC	CV Avg Precision	CV F1
XGBoost (ordinal)	0.859 ± 0.037	0.929 ± 0.017	0.924 ± 0.018	0.840 ± 0.046
XGBoost (one-hot)	0.859 ± 0.034	0.927 ± 0.017	0.924 ± 0.016	0.840 ± 0.043
Random Forest (ordinal)	0.857 ± 0.033	0.926 ± 0.019	0.920 ± 0.024	0.828 ± 0.045
Logistic Regression (one-hot)	0.859 ± 0.024	0.914 ± 0.019	0.901 ± 0.030	0.846 ± 0.027
SVM RBF (one-hot)	0.844 ± 0.013	0.914 ± 0.014	0.900 ± 0.008	0.837 ± 0.018
MLP (one-hot)	0.808 ± 0.026	0.880 ± 0.029	0.872 ± 0.042	0.784 ± 0.032

Table 3: Five-fold cross-validation on the training set. Preprocessing is refit inside each fold.

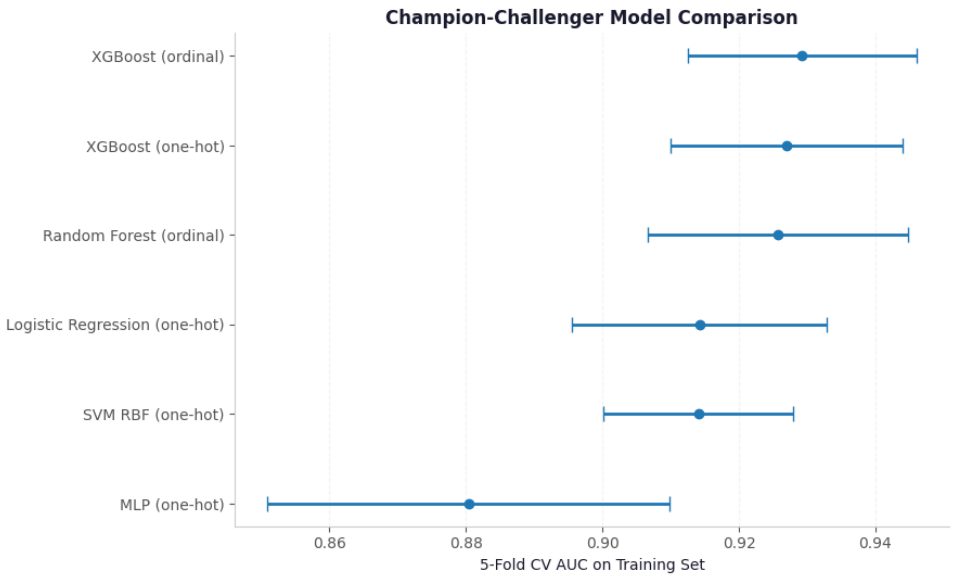

Figure 1: Champion–challenger cross-validation comparison.

3.2 Hold-Out Performance and Calibration

AUC is necessary but not sufficient for credit scoring. A model may rank applicants well while producing probabilities that are too aggressive or too conservative. Since the scorecard and portfolio policy both depend on probability levels, XGBoost is calibrated using isotonic regression. We report Brier score and log loss alongside AUC and KS because calibration quality and tail probability behavior are both relevant in credit applications.

The calibrated model is used for probability-based decisions. SHAP explanations are computed on the raw XGBoost model because tree-level attributions are more direct before probability calibration.

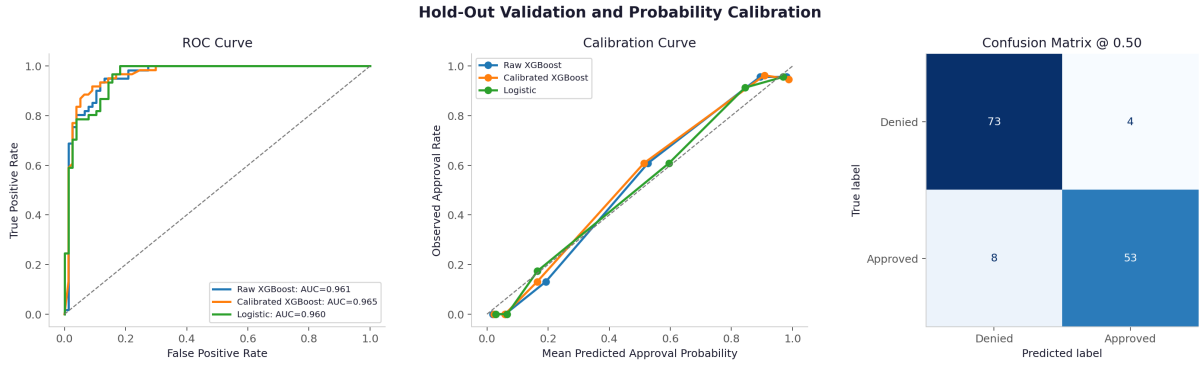

Figure 2: Hold-out validation. The calibration plot compares predicted approval probabilities with observed approval frequencies.

4. Explainability and Scorecard

4.1 Global Explainability

We compute SHAP values on the raw XGBoost champion. Calibration changes the probability scale, but raw tree SHAP remains useful for identifying which features push the model score upward or downward for individual applicants (3).

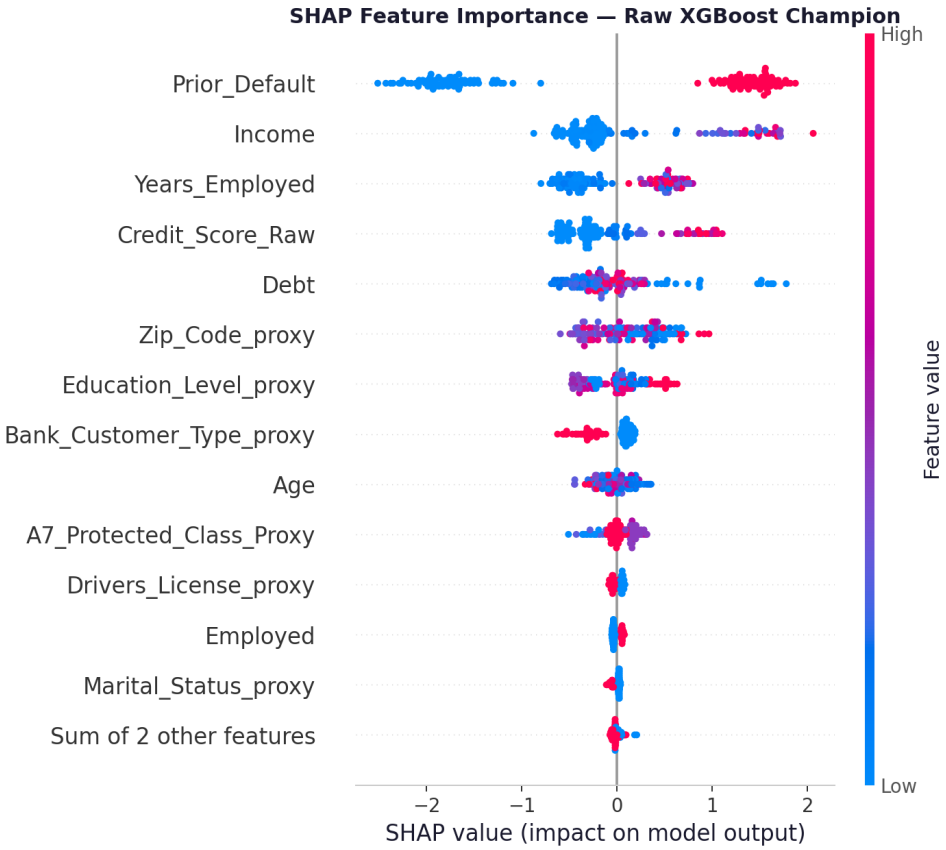

Figure 3: SHAP beeswarm plot for the raw XGBoost champion. Features are ranked by average absolute impact on model output.

The SHAP plot also makes intuitive sense. The most important variables are `Prior_Default`, `Income`, `Years_Employed`, `Credit_Score_Raw`, and `Debt`. These are exactly the types of factors

a lender would normally expect to matter in a credit decision. Applicants with stronger income, longer employment history, better credit scores, and no prior default should generally look safer, while applicants with prior default history or higher debt should appear riskier.

4.2 Individual Reason Codes

For individual decisions, the most negative SHAP contributions are translated into adverse-action-style reason codes. Because the UCI variables are anonymized, these are explanation examples rather than legally final adverse-action notices. In a production setting, reason codes would need verified feature definitions, compliance review, and stability testing.

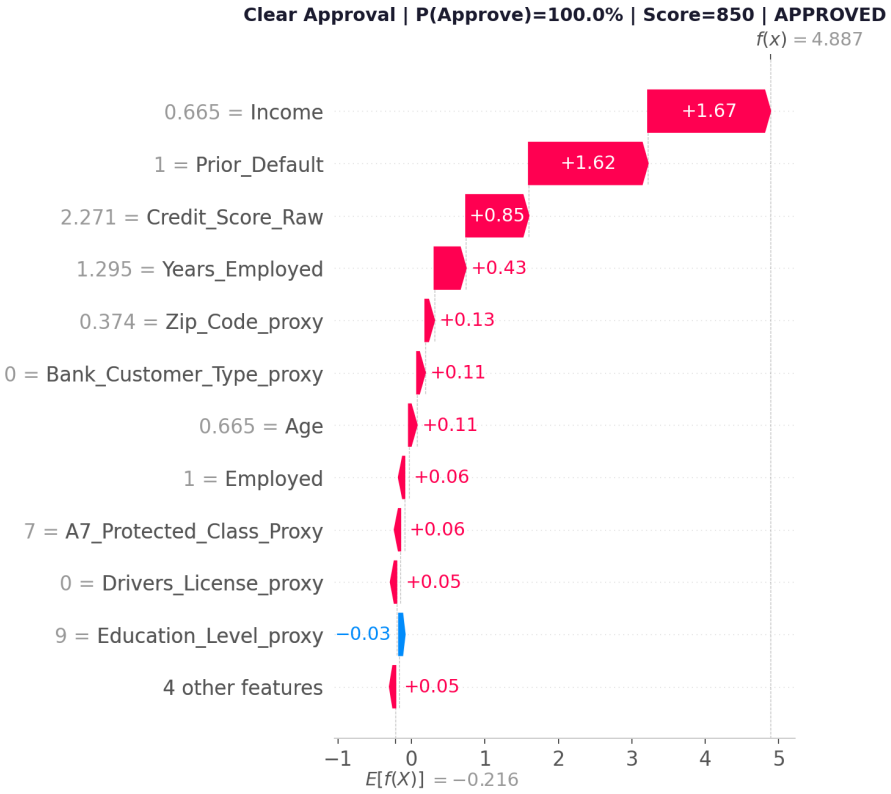

Figure 4: SHAP waterfall — clear approval.

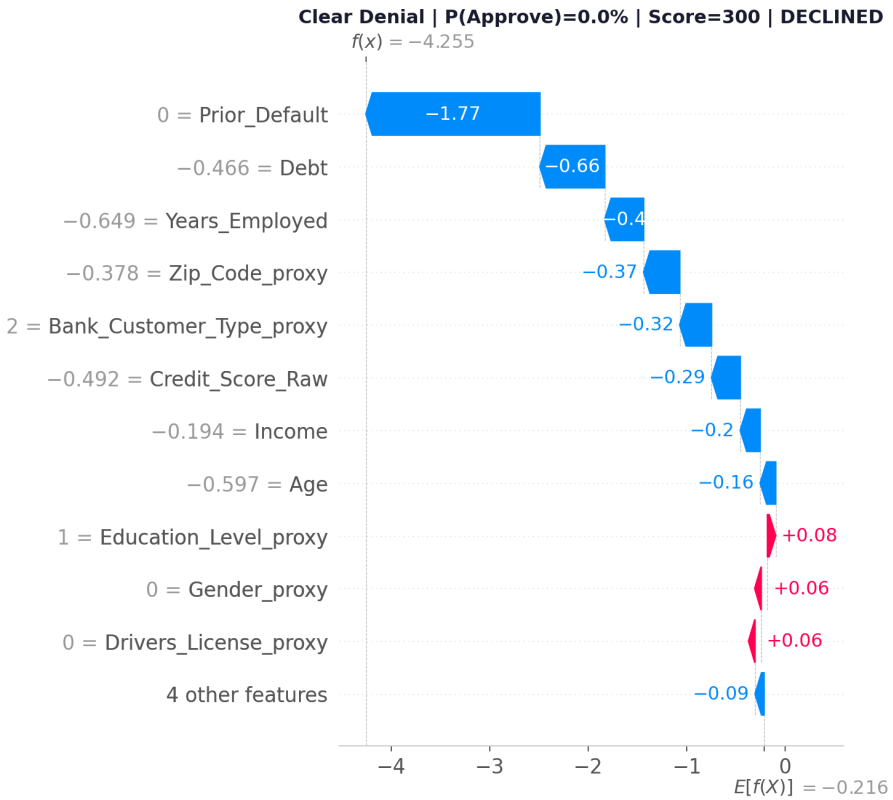

Figure 5: SHAP waterfall — clear denial.

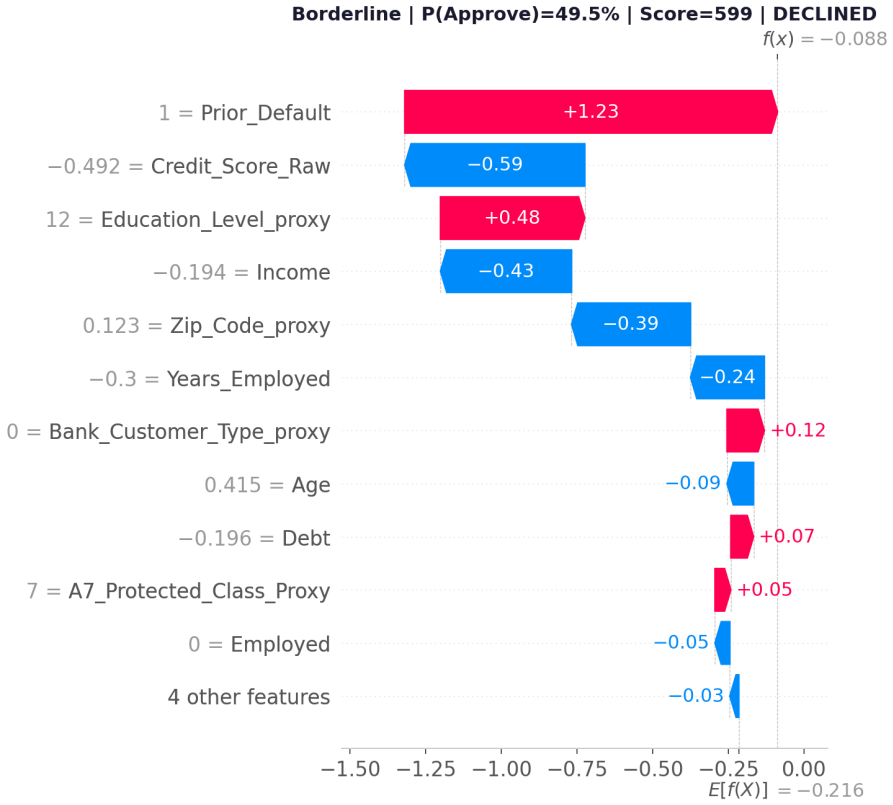

Figure 6: SHAP waterfall — borderline case. All three examples were selected programmatically.

Applicant Type	P(Approve)	Score	Decision	Reason Rank	Feature	SHAP Value
Clear Approval	100.0%	850	APPROVED	1	Education_Level_proxy	-0.034
Clear Approval	100.0%	850	APPROVED	2	Citizenship_proxy	-0.006
Clear Approval	100.0%	850	APPROVED	3	Gender_proxy	-0.002
Clear Denial	0.0%	300	DECLINED	1	Prior_Default	-1.765
Clear Denial	0.0%	300	DECLINED	2	Debt	-0.657
Clear Denial	0.0%	300	DECLINED	3	Years_Employed	-0.395
Borderline	49.5%	599	DECLINED	1	Credit_Score_Raw	-0.594
Borderline	49.5%	599	DECLINED	2	Income	-0.433
Borderline	49.5%	599	DECLINED	3	Zip_Code_proxy	-0.393

Table 4: Example adverse-action-style reason codes generated from individual SHAP values.

4.3 Scorecard Transformation

The calibrated approval probability is mapped to a 300–850 score using a points-to-double-odds transformation. Because the model output is $P(\text{Approve})$, the score must increase with the approval odds:

$$\text{Score} = \text{Offset} + \text{Factor} \cdot \ln\left(\frac{p_{\text{approve}}}{1 - p_{\text{approve}}}\right), \qquad \text{Factor} = \frac{\text{PDO}}{\ln 2}. \tag{1}$$

This sign convention is important. If p were a default probability, the odds term would be inverted. Here, using $\ln((1-p)/p)$ would incorrectly assign lower scores to stronger applicants.

Score Band	Count	Approval_Rate	Avg_Score	Avg_P_Approve
Very Poor (300–499)	14	0.0%	409	1.3%
Fair (500–579)	57	7.0%	533	10.7%
Good (580–669)	34	73.5%	624	66.0%
Very Good (670–739)	24	100.0%	693	95.9%

Table 5: Score-band analysis using calibrated approval probabilities.

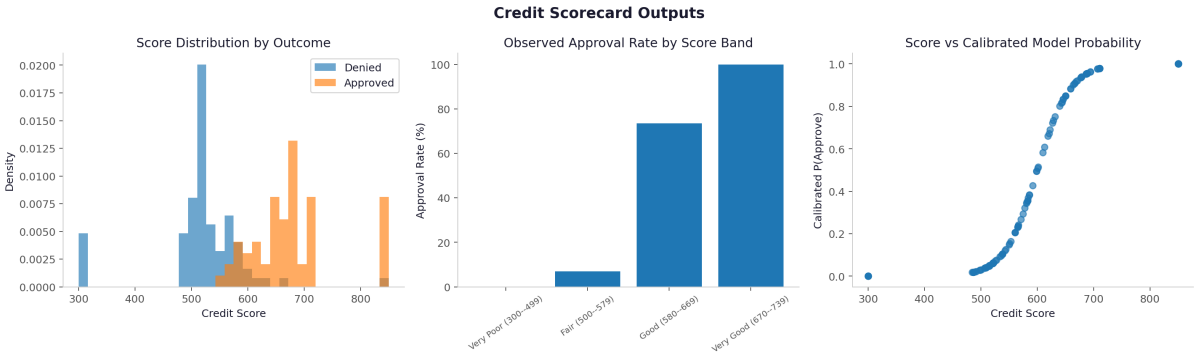

Figure 7: Scorecard outputs: score distribution, observed approval rate by score band, and score–probability relationship.

The score-band table and Figure 7 show that the scorecard behaves as expected. Applicants with low scores have very low approval rates, while applicants with high scores have much higher approval rates. In the Very Poor band, the observed approval rate is 0.0%, and the average calibrated approval probability is only 1.3%. In the Very Good band, the observed approval rate is 100.0%, and the average calibrated approval probability is 95.9%.

Overall, the scorecard is easy to interpret: a higher score means a stronger application and a higher estimated chance of approval. This is important because the scorecard turns the model output into a format that looks more like a traditional credit score.

5. Financial Backtest

5.1 Backtest Design

A flat backtest assumes that every approved applicant receives the same loan amount, APR, and recovery rate. This is easy to explain but unrealistic for credit portfolio management. The revised backtest instead applies a risk-based policy in which score bands determine loan limits, APRs, and recovery assumptions. This converts the model from a binary classifier into a lending-policy tool.

Because the UCI dataset does not include realized repayment, charge-off, or recovery outcomes, the backtest uses the historical approval label as a proxy for credit quality. In the tables, "bad rate" should therefore be read as the share of negative historical approval labels among applicants the model would approve, not as an observed default rate.

Risk Band	Count	Min_Score	Max_Score	Avg_Loan	Avg_APR	Avg_Recovery
Deep subprime	59	300	553	$4,000	27.0%	25.0%
Near-prime	35	621	694	$10,000	13.0%	45.0%
Prime	17	707	850	$12,000	9.0%	55.0%
Subprime	27	561	619	$7,000	20.0%	35.0%

Table 6: Risk-based lending terms assigned by calibrated score band. These are modeling assumptions used for scenario analysis, not observed UCI contract terms.

For an approved applicant, one-period profit is modeled as:

$$\Pi_i = \begin{cases} L_i(r_i - c_f) - c_o, & \text{if applicant } i \text{ is a positive-label applicant,} \\ -L_i(1 - R_i) - c_o, & \text{if applicant } i \text{ is a negative-label applicant,} \end{cases} \tag{2}$$

where L_i is the loan amount, r_i is the APR, c_f is the funding cost, R_i is the recovery rate, and c_o is the operating cost. Portfolio profit is computed over approved applicants and scaled to 10,000 applications.

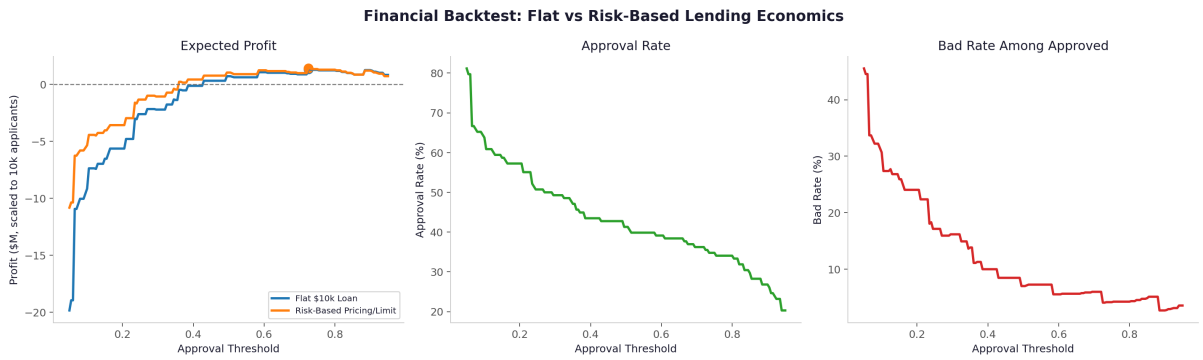

Figure 8: Financial backtest comparing flat-loan economics with risk-based pricing and loan limits.

Policy	Optimal Threshold	Approval Rate	Bad Rate Among Approved	Expected Profit
Flat $10k loan	0.72	35.5%	4.1%	$1,322,464
Risk-based pricing/limit	0.72	35.5%	4.1%	$1,394,928

Table 7: Backtest summary under flat and risk-based economics.

Figure 8 shows the tradeoff in choosing an approval threshold. Higher thresholds approve fewer applicants, but the approved group becomes safer. This is why the approval rate falls while the bad rate among approved applicants also falls.

The best threshold is 0.72 for both policies. At this cutoff, the model approves 35.5% of applicants and has a bad-proxy rate of 4.1%. The risk-based policy earns slightly more expected profit than the flat $10k loan policy: $1,394,928 versus $1,322,464. This suggests that adjusting loan terms by risk band can improve portfolio results.

5.2 Stress Scenarios

To reduce dependence on a single loss-severity assumption, we stress losses on negative-label approved applicants by 25% and 50%. This functions as a simple downturn sensitivity analysis: as losses worsen, the optimal threshold should rise and the approval rate should fall.

Scenario	Default Loss Multiplier	Optimal Threshold	Approval Rate	Bad Rate Among Approved	Expected Profit
Base	1.00x	0.72	35.5%	4.1%	$1,394,928
Mild Stress	1.25x	0.72	35.5%	4.1%	$1,197,464
Severe Stress	1.50x	0.88	26.8%	2.7%	$1,010,870

Table 8: Risk-based backtest under base and stressed loss assumptions.

The stress test shows that when default losses increase, expected profit falls and the model becomes more selective, especially in the severe stress case where the optimal threshold rises to 0.88.

6. Fairness and Regulatory Risk

The most important model-risk question is not whether XGBoost slightly outperforms logistic regression. It is whether the model relies on variables that may proxy for protected characteristics. Since A7 is sometimes interpreted as an ethnicity-like variable but is not verified, we treat it as a protected-class proxy candidate and run an ablation test.

Model	AUC	KS	Optimal Threshold	Expected Profit	Max A7 Approval Gap
Calibrated XGBoost with A7	0.965	0.827	0.72	$1,394,928	65.8%
XGBoost excluding A7	0.960	0.805	0.80	$1,532,609	57.9%

Table 9: A7 proxy ablation. If the performance loss from excluding A7 is small relative to the governance benefit, a production-style recommendation would favor the model excluding A7.

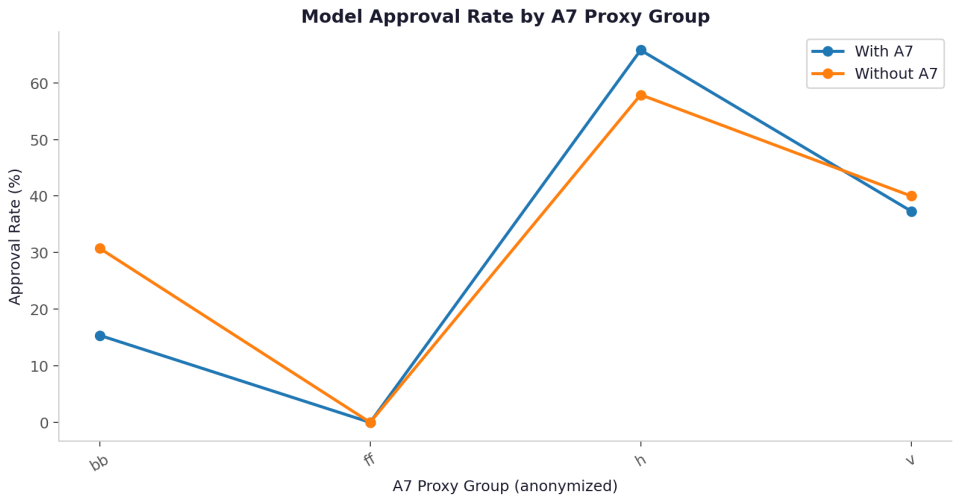

Figure 9: Approval-rate differences across A7 proxy groups with and without A7.

The A7 test shows that removing A7 causes only a small drop in model performance, while reducing the approval-rate gap across A7 groups. This suggests that a production model should likely exclude A7 to reduce fairness and regulatory risk.

7. Risks and Limitations

Small sample size. The dataset has only 690 observations, so hold-out metrics are sensitive to sampling variation. The analysis should be interpreted as a model-development exercise, not a production validation.

No time dimension. The dataset is cross-sectional and does not include application dates, origination dates, macroeconomic conditions, or performance windows. As a result, we cannot perform true out-of-time validation, vintage analysis, population stability monitoring, or model-decay analysis.

Approval label rather than repayment outcome. The target is historical approval or denial, not realized borrower performance. A lender would need repayment data, reject-inference correction, or randomized/exploratory approvals before treating this as a default-risk model.

Anonymized variables. Feature names and category meanings are obscured. Reason codes, feature labels, and fairness conclusions are therefore approximate and should be treated as methodological demonstrations.

Backtest assumptions. Loan limits, APRs, recovery rates, funding cost, and operating cost are scenario assumptions. They make the economics more realistic than a flat-loan backtest, but they are not observed in the UCI dataset.

8. Recommendation

We recommend presenting calibrated XGBoost as the research champion because it combines strong rank-ordering performance, usable probability calibration, SHAP-based explanation, and favorable portfolio economics under the risk-based backtest. For a production-style recommendation, however, the safer candidate is not automatically the highest-AUC model. Deployment should proceed only after validating or excluding A7, replacing the approval label with observed performance outcomes, correcting for reject inference, and testing stability on a larger

time-stamped portfolio.

9. Source Attribution and Generative-AI Disclosure

Dataset and Library Sources

The UCI Credit Approval dataset is cited as (2), with the original decision-tree methodology attributed to (1). SHAP values are computed using the `shap` Python library and cited as (3). Scorecard construction follows the points-to-double-odds framework described in Siddiqi (2006) and Anderson (2007). All performance metrics, figures, and tables are generated by the project notebook and are not copied from external sources.

LLMs Used

The following large language model systems were used during this project:

- **Claude Sonnet 4.6** (Anthropic, accessed via `claude.ai`) — primary assistant for pipeline architecture, code review, SHAP reason codes, and report editing.
- **ChatGPT** (OpenAI, accessed via `chatgpt.com`) — used for calibration methodology, back-test design, and scorecard formula verification.
- **DeepSeek** (DeepSeek, accessed via `chat.deepseek.com`) — used for cross-checking Python implementations and debugging.
- **Gemini** (Google, accessed via `gemini.google.com`) — used for literature-search assistance and report-structure feedback.

Specific Prompts and Uses

The table below documents the prompts submitted to LLMs, the model used, and how the output was incorporated. All LLM output was reviewed, corrected, and edited by the team before inclusion. No quantitative result was taken directly from an LLM response.

Prompt (paraphrased)	Model	How output was used
"Design a sklearn Pipeline for credit scoring that avoids data leakage during cross-validation, with median imputation, standard scaling, and ordinal encoding for tree models."	Claude Sonnet 4.6	Provided the initial Pipeline skeleton. The team corrected the `ColumnTransformer` column selection and added a separate one-hot branch for logistic regression.
"How do I calibrate XGBoost probabilities using isotonic regression in sklearn? What metrics confirm calibration quality?"	ChatGPT	Suggested `CalibratedClassifierCV(method='isotonic')` and recommended Brier score, log loss, and calibration curves. The team implemented the calibration plot and verified metrics in the notebook.
"Write a risk-based lending backtest in Python. Score bands should determine loan limits, APRs, and recovery rates. Scale to 10,000 applicants."	Claude Sonnet 4.6	Produced a draft profit function. The team restructured it with vectorized `numpy`, corrected the loss sign, and added stress scenarios independently.
"Generate SHAP waterfall reason codes as adverse-action-style text using the top three negative SHAP features for individual applicants."	DeepSeek	Provided the reason-code formatting logic. The team adapted variable names and verified SHAP values against model output.
"Review this LaTeX credit report for structure and alignment with investment-bank-style credit analysis. What sections are missing?"	Gemini	Suggested Executive Summary, Risks and Limitations, and Recommendation sections. The team wrote and edited the section content.
"What is the correct sign for the points-to-double-odds scorecard when the model outputs P(Approve) rather than P(Default)?"	ChatGPT	Confirmed that $\ln(p/(1-p))$ with $p = P(\text{Approve})$ correctly increases scores for stronger applicants. The team verified this before including the formula.
"What default-loss multipliers are typically used in bank credit downturn stress tests?"	Claude Sonnet 4.6	Suggested 25% and 50% loss-severity shocks as simple stress proxies. The team implemented and checked the threshold-sensitivity results in the notebook.
Make the latex more easy to read	ChatGPT	Compared the modification that were made and improved each paragraph.

Table 10: LLM prompts, models used, and how each output was incorporated and verified by the team.

All quantitative claims in this report—AUC, KS, Gini, Brier score, optimal threshold, approval rate, bad-proxy rate, and expected profit—are generated by the project notebook and verified by the team. No quantitative result was taken directly from LLM output.

References

[1] Quinlan, J.R. (1987). Simplifying decision trees. *International Journal of Man-Machine Studies*, 27(3), 221–234.

[2] UCI Machine Learning Repository (1994). Credit Approval Data Set. https://archive.

ics.uci.edu/dataset/27/credit+approval.

[3] Lundberg, S.M., & Lee, S.I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30.

[4] Siddiqi, N. (2006). *Credit Risk Scorecards: Developing and Implementing Intelligent Credit Scoring*. John Wiley & Sons.

[5] Anderson, R. (2007). *The Credit Scoring Toolkit*. Oxford University Press.